

多框架模型接入与统一 IR 语义规范

从 PyTorch / TensorFlow / ONNX 模型接入，到 Core ATen 统一表示、可执行语义契约，再到 FlagTree XPU 阶段性验证的完整进展。

三种模型入口

Core ATen 统一图

动态 Shape

结构化控制流

逐算子语义规则

XPU 实机证据

汇报口径

功能模块一已完成代表性端到端闭环；功能模块二已形成可运行原型并进入规模化验证。两者均不能被解释为“全部模型、全部算子、全部 Shape、全部后端已经 100% 支持”。

一页看懂：本阶段交付了什么

核心变化不是“又支持了一个模型”，而是建立了一条可复用的工程链路：先把不同框架收敛为统一模型图，再用机器可执行的规则确认转换前后的语义没有悄悄改变。

模型入口

3 类PyTorch · ONNX ·
TensorFlow

现有样本语义登记

39 / 39

当前 22 个去重 ONNX 模型

真实模型规则执行

86%+

两模型均 0 条规则失败

当前阶段

验证中

远未等同整体 100%

功能模块一

模型接入与统一表示

解决“不同框架的模型怎样进入同一条编译链”。三种入口汇入可保存、可回读的 ExportedProgram / Core ATen，并保留结构、类型、Shape 和控制流信息。

- 中型动态 Shape 与结构化控制流已经有 XPU 证据。
- 代表性模型闭环完成，模型规模化兼容仍需继续扩展。



功能模块二

统一 IR 语义规范

解决“图虽然统一了，但算子、dtype、Shape 和 layout 的含义是否一致”。引入版本化注册表、语义契约和逐算子可执行规则。

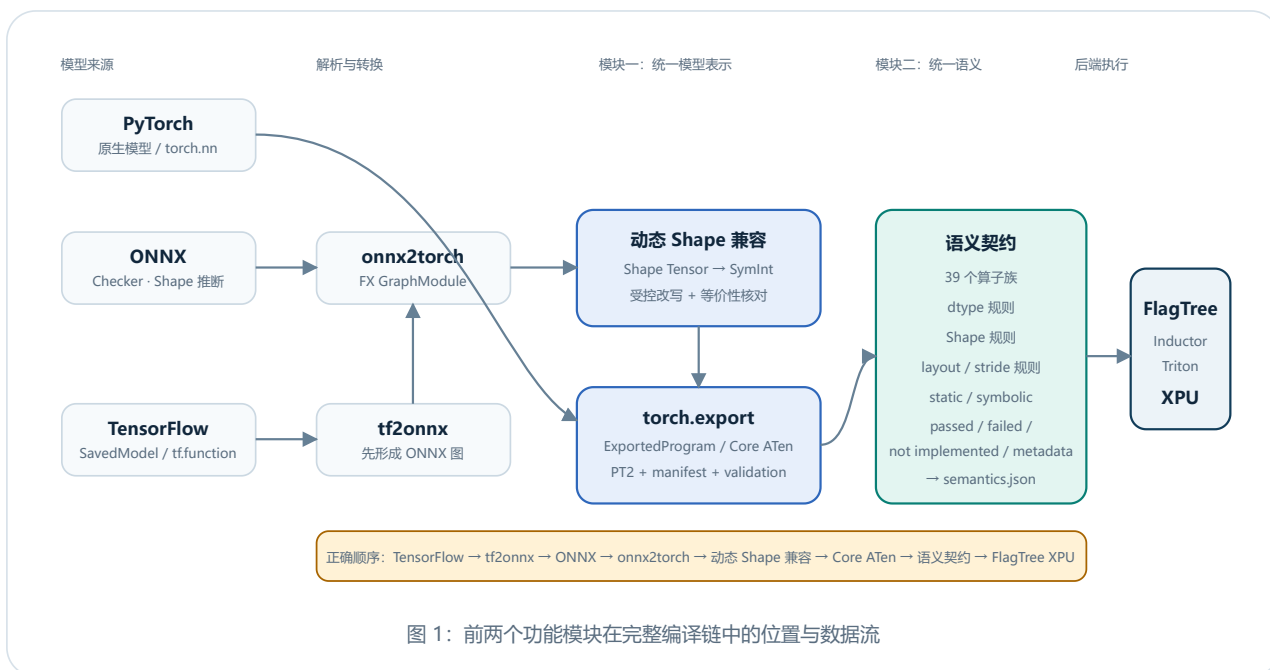
- 小图 6/6 规则执行通过，两类真实模型执行比例达到 86% 以上。
- 尚需补齐符号 reshape、矩阵布局和运行时 Shape 元数据。

最重要的管理结论

当前已经从“单点脚本验证”推进到“统一产物 + 自动语义检查 + XPU 复验”的可复用链路；下一阶段的重点应从继续堆演示样例，转向 250/500 模型的分层统计、缺口聚类与规则补齐。

Big picture：两个模块怎样首尾衔接

模块一负责“把模型送进来并统一形态”，模块二负责“把统一后的含义说清楚并检查出来”。模块二直接消费模块一生成的 Core ATen 与 manifest，因此两者是前后相接，而不是两套独立工作。



职责边界

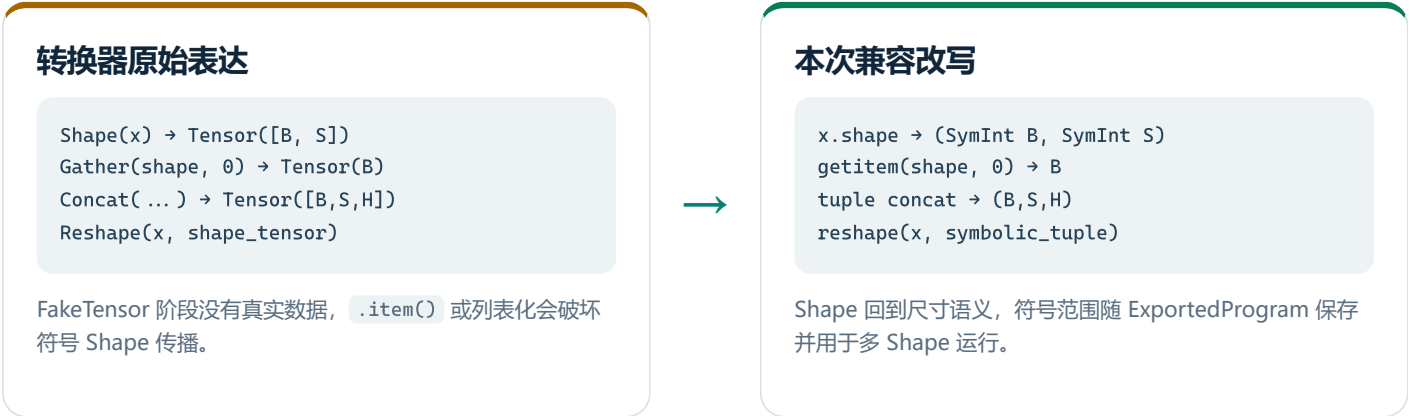
Core ATen 是模型级统一表示；FlagTree / Triton 仍负责后续 Kernel 级 Lowering、优化和设备代码生成。语义注册或 XPU 跑通，不等于所有算子都已生成 FlagTree 自有 Kernel。

功能模块一：多框架接入与统一 Core ATen

目标是将 PyTorch、ONNX、TensorFlow 的模型结构、类型、Shape 与控制流信息，汇入同一种可保存、可回读、可核对的模型级中间表示。



为什么中型模型此前会卡住



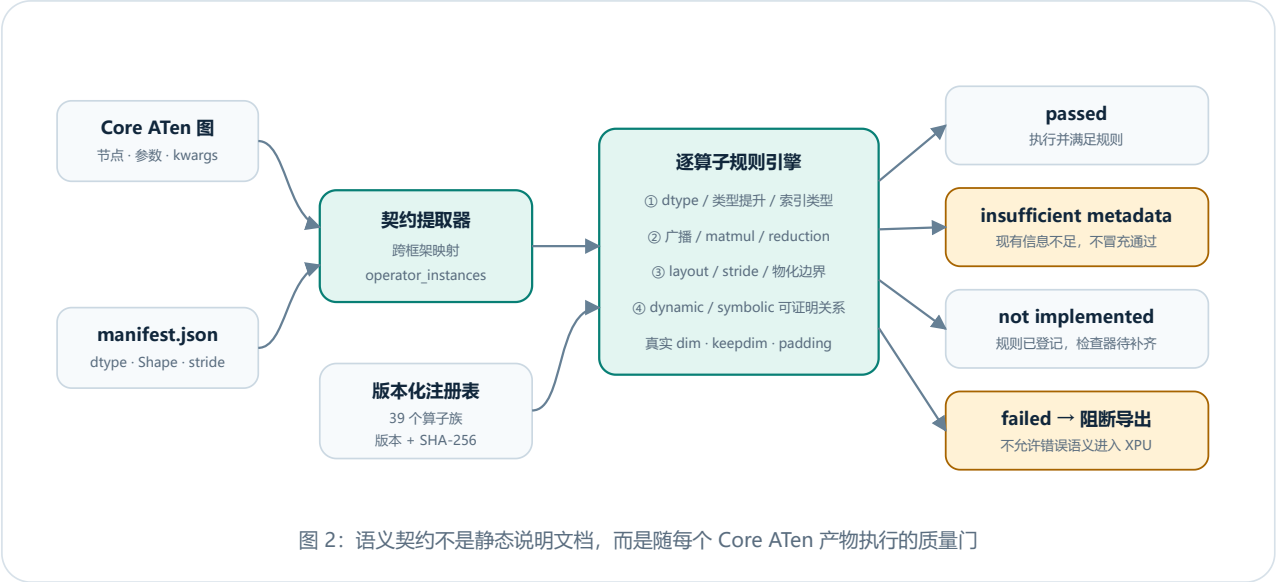
代表性闭环	统一产物	XPU 方式	结果
ONNX DistilBERT	单一动态 PT2, batch / sequence 符号范围	<code>torch.compile(fullgraph=True)</code> ，复用一个 callable	1×2、2×4、3×7 通过
TensorFlow DistilBERT	tf2onnx 后进入同一 ONNX/Core ATen 路径	同一 PT2 跨三组 Shape	最大误差约 1.53e-5
ONNX If / Loop / Scan	结构化高阶算子与控制流契约	8 组代表性图 fullgraph	CPU/XPU 最大误差 0
EmbeddingGemma-300M	动态 Core ATen/PT2	STS12 共 3,108 对文本	主分数相对差 0.00007697%

阶段判断

模块一的代表性功能闭环已经建立，但“任意第三方模型兼容率”“全部 ONNX 算子”“长序列和复杂生成模型”“规模化性能”仍是后续工作，不使用整体 100% 的表述。

功能模块二：把“图能转换”升级为“语义可证明”

仅仅看到算子名称并不足以证明转换正确。比如同样叫 reshape，是否保持元素总数；同样叫 reduce，dim 与 keepdim 是否正确；同样叫 expand，stride 是否体现广播，这些都需要机器可执行的规则。



注册表

版本 2026.09.22，内容哈希固定；定义算子族、dtype、Shape、layout 和扩展命名空间，便于判断两个产物是否依据同一套规范。

精确参数

operator_instances 保存普通参数和 kwargs，因此检查器能看到真实的 dim、keepdim、permutation、padding 和 slice bounds。

诚实覆盖率

把已执行、失败、尚未实现、元数据不足分开统计。只有真正运行过检查器的规则才进入执行比例，避免用“登记覆盖率”代替完成度。

本轮新增规则覆盖面

类别	已经能够执行的代表性检查	典型问题
dtype	同类型提升、除法浮点路径、累加类型、Where 条件、索引整数、Cast 目标类型、Dropout mask	防止静默窄化、错误索引类型和转换目标丢失
Shape	符号广播、matmul、transpose、concat/stack、expand/repeat、slice、embedding/gather、reduction、linear、pad、creation	防止轴、广播、收缩维和动态范围被误改
layout	保持或广播、物化输出、reshape stride、transpose stride、expand 零 stride、view/物化边界	防止只看 Shape 一样，却忽略存储布局语义

一个具体例子

aten.mean.dim 的 keepdim=True 实际位于 kwargs。旧做法只根据输入输出 Shape 猜测，容易误判；现在审计器读取真实调用参数，再按 dim 和 keepdim 推导期望 Shape。动态 slice 的 end 如果来自运行时 SymInt，则明确记录为“元数据不足”，而不是为了提高覆盖率强行判为通过。

证据：测试覆盖、真实模型与 XPU 结果

这里同时给出三类数字：注册表是否认识算子、规则检查器是否真正执行、模型是否经过 XPU。三类指标互相补充，但不能互相替代。

规则真实执行比例

小型 ONNX Add + Relu		6 / 6
PyTorch 结构化控制流		83.33%
all-MiniLM-L6-v2		88.06%
bge-m3		86.58%

图中比例只统计实际执行的规则；未实现或元数据不足不会被算作通过。

自动回归

模型接入完整回归

FlagTree 语义专项

24 passed

15 passed

PyTorch / ONNX / TF / 控制流

注册表 / 契约 / 规则审计

XPU 小图验收

6/6 规则通过，`torch.compile(fullgraph=True)` 通过，CPU/XPU 最大绝对误差为 0。

Mini

all-MiniLM-L6-v2

547 个 Core ATen 算子实例，1,641 条规则检查；1,445 条实际执行并通过，0 条失败。单一动态 PT2 已跨三组 Shape 在 XPU 复用。

执行 1,445

失败 0

比例 88.06%

XPU 误差 $\leq 5.72e-6$

BGE

BAAI/bge-m3

1,893 个 Core ATen 算子实例，5,679 条规则检查；4,917 条实际执行并通过，0 条失败。约 2.27 GB 动态 PT2 全图编译通过。

执行 4,917

失败 0

比例 86.58%

sentence 误差 $\leq 5.50e-5$

验证层级	当前证据	能证明什么	不能证明什么
语义注册覆盖	22 个去重模型、20,659 节点、39/39 种 ONNX 算子已登记	当前样本中的算子能映射到已定义语义族	不等于转换、Lowering 或 Kernel 已实现
可执行规则覆盖	真实模型 86.58%–88.06%，0 failed	大部分逐节点 dtype/Shape/layout 检查已真实运行	剩余规则仍需实现或补充元数据
XPU 模型执行	DistilBERT、MiniLM、bge-m3、EmbeddingGemma 与控制流代表图	指定图和指定 Shape 能在现有 XPU 环境运行并与参考对齐	不等于 250/500 模型全量通过或性能达标

边界与风险：为什么现在不能说 100%

本阶段的价值在于把缺口变成可测量、可定位的工程问题，而不是用少量跑通案例覆盖所有未知范围。

已经有证据的能力

- 三种入口汇入同一 Core ATen/PT2 产物。
- 动态 batch / sequence 与结构化 If/Loop/Scan 有代表性 XPU 结果。
- 语义契约随产物落盘，错误规则能够阻断导出。
- 39 个算子族覆盖当前 22 个去重模型的语义登记。
- 两类真实模型可执行规则比例达到 86% 以上且无失败。
- 代码已提交并推送到 `origin/dev/hx`。

仍需继续完成的能力

- 尚未取得并扫描全部 250/500 模型。
- 符号 reshape 元素数量等价证明仍是最大元数据缺口。
- 矩阵逻辑轴 layout、运行时 Shape 查询规则尚待实现。
- CNN、检测、生成式模型和长序列动态范围覆盖不足。
- PyTorch 2.5 与 2.9 的 PT2 序列化兼容尚未统一。
- 语义覆盖、转换覆盖、Core ATen 导出和 XPU 执行需要分别统计。

禁止混用的三个概念

“注册表认识这个算子” ≠ “onnx2torch 能正确转换” ≠ “FlagTree/XPU 已有稳定高性能 Kernel”。汇报时必须分别说明语义层、转换层和后端执行层的证据。

语义覆盖

算子规则是否定义

转换覆盖

前端能否变成 Core ATen

导出覆盖

动态图能否保存回读

执行覆盖

XPU 是否正确、稳定、够快

下一步建议：从单模型成功走向规模化可交付

下一阶段不建议继续只用“又跑通一个模型”衡量进展，而应建立分层漏斗：每个模型在哪一层失败、失败原因是什么、修复能覆盖多少同类模型。

P0

补齐高频规则

优先实现符号 reshape 元素数量证明、矩阵逻辑轴 layout、运行时 Shape 查询元数据。它们覆盖当前大多数未判定项。

P0

批量审计 250/500

直接复用现有 PT2 审计工具，形成语义、转换、导出、XPU 四张独立清单，不再把一个百分比混作全部完成度。

P1

扩展模型类型

从 BERT 类扩展到 CNN、检测、生成式模型；补充复杂 Slice/Range、多输出 Shape 与更长 sequence 边界。

P1

性能与版本工程

定位 20 分钟级编译慢点，建立 PT2 版本策略；CI 暂缓，但保留脚本化复验入口，后续再自动化。

推荐汇报指标

指标	建议分母	当前示例	下一阶段目标方式
语义注册覆盖率	模型集中出现的唯一 ONNX 算子	当前样本 39/39	扩展到完整模型清单后重新统计
规则执行比例	Core ATen 算子实例 × 三类规则	MiniLM 88.06%；bge-m3 86.58%	按缺口类型逐轮提升，不掩盖 metadata
转换成功率	进入 onnx2torch/tf2onnx 的模型数	单独记录	按失败算子族聚类
XPU 正确执行率	成功导出 Core ATen 的模型数	单独记录	同时记录精度、运行时间和编译时间

建议的一句话总结

“我们已经打通三框架到 Core ATen、再到语义契约和 XPU 的代表性闭环；当前重点是把这条链路规模化到完整模型集，并用分层指标逐步补齐真实缺口。”

代码、产物与复验入口

GitHub 留存

仓库： Xproject-CHKU-SZ/FlagTree

分支： dev/hx

最新提交：
ccd65da3314d82788f66093b0fe83d550134dc26

提交说明： Expand executable model IR semantic validation

该提交包含规则引擎扩展、契约参数保留、注册表更新、PT2 审计工具、单元测试和说明文档。

核心源码

```
python/flagtree_model_ir/registry.py
python/flagtree_model_ir/contract.py
python/flagtree_model_ir/rule_checks.py
model_import_adapters/scripts/audit_core_aten_semantic_rules.py
python/test/unit/tools/test_model_ir_rule_checks.py
docs/model-ir-semantics.md
```

每个模型的标准产物

```
model.core_aten.pt2
manifest.json
semantics.json
validation.json
graph.readable.txt
graph_module.py
example_inputs.npz
```

服务器主要证据目录

```
artifacts/module2_executable_rules_20260922/
server_test_results/top500_all_minilm_l6_v2_20260915/
server_test_results/top500_bge_m3_20260915/
artifacts/control_flow_xpu/
server_test_results/embeddinggemma_300m_onnx_20260911/
```

复验注意

PyTorch 2.5 生成的旧 PT2 不能保证被 PyTorch 2.9 直接加载；XPU 复验应使用与产物匹配的运行时，或在目标版本重新导出。连接凭据不进入报告和 Git 仓库。